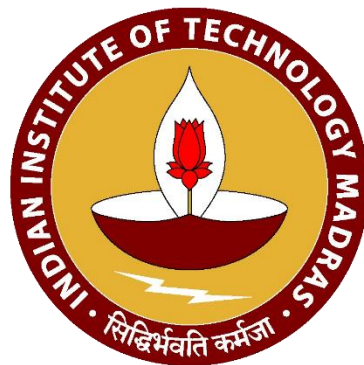


DA5400: Foundations of Machine Learning

Topic 8: Calibration & Decision Thresholding In Classification



Contents

- Motivation for Classifier Calibration
- What is Calibration?
- Common causes of Mis-calibration
- Types of Mis-calibration
- Ways to Assess Calibration of a Classifier
- Re-calibration of a Classifier
- Decision Thresholding

Motivation for Classifier Calibration

- Recall:
 - Scores:
 - Classifiers output a score $s(x)$ for each sample and many times it is $P(y = 1|x)$
 - Alternately, it could be distance to decision boundary, log-odds, etc.
 - Ranking: Order after sorting by score
 - If sorted by $P(y = 1|x)$, we get ranked list of most likely positive to least likely positive samples
- Issue: A classifier may rank cases correctly, yet its output scores may not be reliable probabilities.

Motivation for Classifier Calibration

- Eg: A bank wants to predict whether a loan applicant will default or not default within 12 months
- Class 1: Default; Class 0: No default
- Classifier gives a risk score $s(x)$ for each applicant
- Q: Do these scores give sensible ranks?
- A: Yes
- Q: Are these scores reliable probabilities?
- Suppose we examine many past applicants with similar predicted scores and find
 - Among applicants with score near 0.90, only about 60% actually default
 - Among applicants with score near 0.80, only about 55% actually default
 - Model scores are too high if interpreted as probabilities and are not trustworthy
- In many real-world applications, decisions may depend on predicted probabilities as well and overconfident scores impact such decisions

Applicant	True Class	Score
A	1	0.9
B	1	0.8
C	0	0.7
D	0	0.4

When Probabilities are Needed

- Predicted probabilities are needed when decisions depend not just on the predicted class, but on **how likely** the event is
 - In **medical screening**, a patient may be referred for further testing only if the predicted disease risk is sufficiently high
 - In **fraud detection**, transactions may be ranked by predicted fraud probability so that investigators examine the riskiest ones first
 - In **credit risk**, a loan may be rejected only if the predicted probability of default makes the expected loss too high
 - In **marketing**, a customer may be contacted only if the predicted probability of conversion is high enough to justify the campaign cost
- **Key Point:** When decisions are cost-sensitive or resource-constrained, calibrated probabilities are more useful than raw class labels

What is Classifier Calibration?

- A classifier is well calibrated if:
 - among all cases predicted with probability s ,
 - roughly $s\%$ actually, belong to class 1
- Mathematically:

For each score range,
average predicted probability $\approx$ observed fraction of positives
- In simple words, calibration is about the accuracy of predicted probabilities
- Note:
 - Metrics like accuracy, F1, precision, recall, AUC mainly evaluate discrimination (ability to separate positives from negatives)
 - They do not guarantee probability calibration

Common Reasons for Mis-Calibration

1. The model is trained to separate classes, not to produce accurate probabilities
 - Many classifiers are optimized to improve classification or ranking performance, not calibration
 - **Example:** A fraud model may correctly give higher scores to fraudulent transactions than genuine ones, but a score of 0.9 may not really mean a 90% fraud chance
2. Overfitting can make probabilities too extreme
 - When a model fits the training data too closely, it may become overconfident on new data.
 - **Example:** A decision tree may learn very specific patterns and output probabilities close to 0 or 1, even though the true outcomes are much less certain.

Common Reasons for Mis-Calibration

3. Strong regularization can shrink probabilities
 - Regularization may reduce overfitting, but sometimes it also makes the model less confident than it should be
 - **Example:** A logistic regression model with strong regularization may predict many cases around 0.4 to 0.6 probability, even when some of them are much more likely to be positive
4. Class imbalance can distort probability estimates
 - When one class is much rarer than the other, the model may learn scores that rank well but do not reflect the true event probabilities
 - **Example:** When the disease is rare, the model may still identify the riskiest patients correctly, but a score like 0.40 may not mean that 40% of such patients truly have the disease

Common Reasons for Mis-Calibration

5. Some model families naturally produce poor probabilities
 - Certain classifiers such as naïve bayes, decision trees, boosted trees produce scores that are useful for ordering cases, but not directly reliable as probabilities
 - **Naive Bayes** assumes conditional independence of features, which is often violated, making probabilities too extreme
 - **Decision trees** estimate probabilities from the class proportions in terminal leaves, which can be unstable when a leaf contains only a small number of samples
 - **Boosted trees** focus on correcting classification errors stage by stage resulting in overconfident probabilities

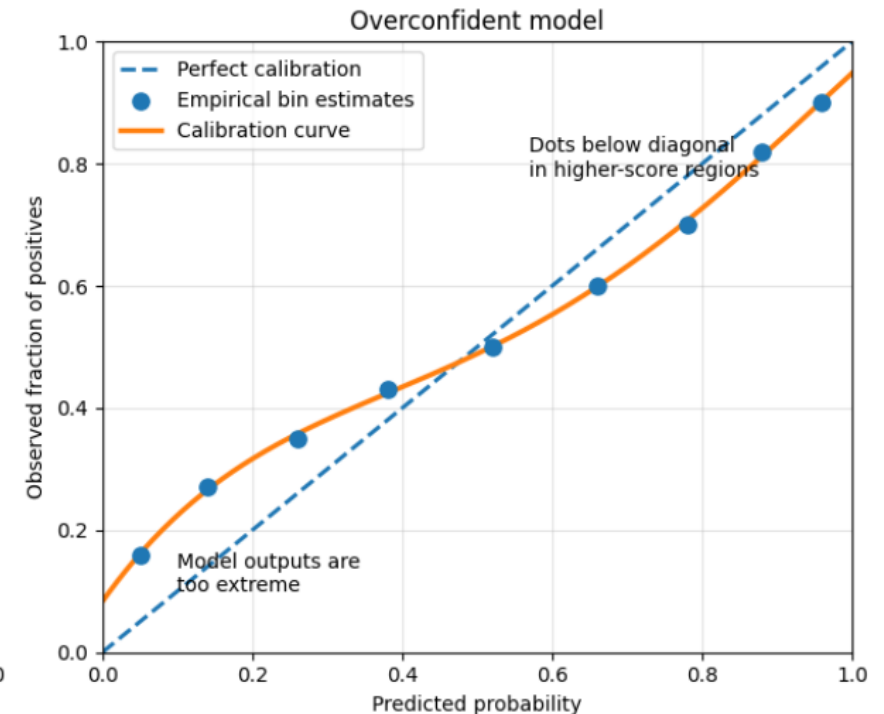
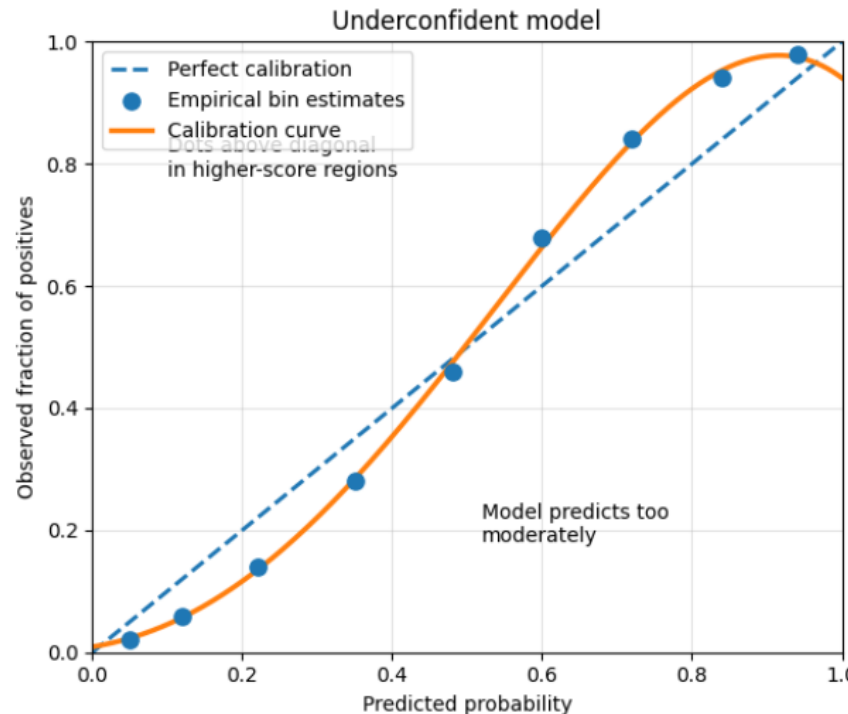
Common Reasons for Mis-Calibration

6. Distribution shift can make a previously calibrated model become miscalibrated
 - A model may be well calibrated on old data but not on new data if the population changes
 - **Example:** A credit risk model calibrated using applicants from one year may become miscalibrated later if customer behavior or economic conditions change
 - **Bottomline:** Miscalibration can arise from the learning objective, model complexity, data imbalance, model type, or changes in the data distribution.

Types of Mis-calibration

- **Overconfident model:** predicted probabilities are too extreme predicts 0.95 when true frequency is 0.75
- **Underconfident model:** predicted probabilities stay too close to the middle predicts 0.60 when true frequency is 0.80

- **Underconfident:** curve tends to lie above the diagonal for higher probabilities - observed frequencies are higher than predicted ones
- **Overconfident:** curve tends to lie below the diagonal for higher probabilities - predicted probabilities are too large



Calibration Plot / Reliability Diagram

Ways to Assess Calibration of a Model

1. Brier Score: $\frac{1}{N} \sum_{i=1}^N (\hat{s}_i - y_i)^2$ *y_i : class label $\{0, 1\}$*

- Consolidated score as to how close predicted probabilities are to actual outcomes
- **Note:** Brier score alone does not tell **where** miscalibration occurs
- Can be used to compare a model before and after re-calibration

2. **Reliability Diagram/Calibration Plot**

- Plot of Observed fraction of Positives Vs Predicted Probability
- Can be used to exactly diagnose the miscalibration of a model

How is a calibration plot constructed?

Step 1: Obtain predicted probabilities from the classifier on validated set



Step 2: Group predictions into bins



Step 3: For each bin, compute two quantities:
1) Average predicted probability (Score) 2) Observed fraction of positives



Step 4: Plot each bin as one point (Avg. prob, Obs frac. of positives)

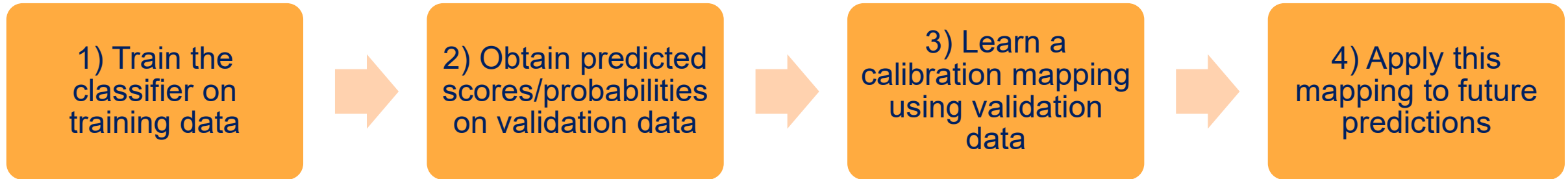


Step 5: Compare with diagonal line to determine whether the model is under-, overconfident or well calibrated

Re-Calibration of a Classifier

Re-Calibration of a Classifier

- Typical Workflow



- Common Approaches for Calibration Mapping

1. Platt scaling – parametric logistic mapping
2. Isotonic regression – nonparametric monotonic mapping
3. Temperature scaling – rescales logits, common in deep learning

Platt Scaling

- Platt scaling learns a **logistic transformation** that converts raw classifier scores into calibrated probabilities
- **Idea:**
 - Suppose a classifier gives a score S
 - Instead of directly treating S as probability, we fit a logistic regression model which returns calibrated probability

$$P(y=1|S) = \frac{1}{1 + e^{A+B}}$$

A, B : parameters

- **Interpretation:**
 - This learns a sigmoid mapping so that
 - overly extreme scores are compressed
 - underconfident scores are stretched
 - Probabilities are closer to observed frequencies
- **Limitation:** Assumes miscalibration can be corrected by a logistic curve

Isotonic Regression

- Isotonic regression learns a **monotonic mapping** from score to probability
- **Idea:** It learns $p_{\text{cal}} = f(s)$ such that $s_1 < s_2 \Rightarrow f(s_1) \leq f(s_2)$
- It implies:
 - higher scores must still correspond to higher probabilities
 - but the exact shape is learned from validation data
 - Mapping adapts to fix the observed calibration error
- Typically solves an optimization problem:

$$\text{Min}_{f_1, f_2, \dots, f_N} \sum_{i=1}^N (y_i - f_i)^2 \quad \text{s.t.} \quad f_1 \leq f_2 \leq f_3 \leq \dots \leq f_N$$

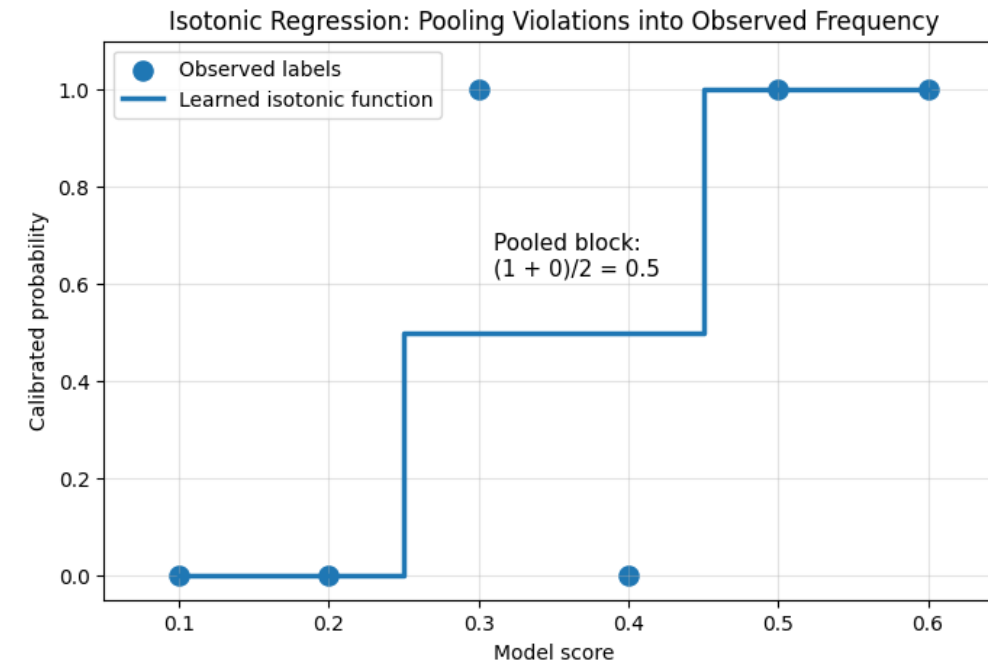
- More flexible than Platt scaling in correcting complex miscalibration patterns
- **Limitation:** Overfits on small validation data

Isotonic Regression: Example

- **Initial labels:** 0,0,1,0,1,1 are not monotonic
- Typically, the violating block is replaced by its average (observed frequency)
- **Interpretation:**
 - Piecewise constant function which maps
 - Low scores to 0
 - Middle scores to 0.5
 - High scores to 1
- **Intuition:** In isotonic regression, the calibrated value assigned to a pooled score region is the observed fraction of positives in that region

Score	Label
0.10	0
0.20	0
0.30	1
0.40	0
0.50	1
0.60	1

Sorted Validation Data



Decision Thresholding

Motivation: Calibration is Not Enough

- Good probabilities do not automatically give the best decision
- Even after calibration, we still need a rule for converting probabilities into actions
- This requires a suitable decision threshold
- **Issue:** Default threshold of 0.5 is not always optimal. It is suitable only when:
 - Both classes are equally important
 - False positives and false negatives have similar cost
 - Business objective is close to minimizing classification error
- In practice:
 - false negatives may be much more costly – Missing to predict a loan that would default
 - false positives may consume expensive resources – Additional screening of a non-existing disease
 - profit may depend on more than correctness

Case Study on Loan Risk Assessment

- **Business challenge:** First-time borrowers had limited credit history - the key challenge was to reduce risk without rejecting too many genuine borrowers
- **Objective:** Maximise the net gain or profit over all loans
- **Modelling Approach:**
 - Built a classification model which predicts probability of repayment of loans on time, with moderate delay, with lot of delay and default
 - Probabilities are to calculate the expected loss and then expected net gain (using revenue)
- **Profit Aware Calibration:**
 - Re-calibration function was learnt to adjust the probability scale so that the total expected net gain is maximized over the validation set
 - Calibrated probabilities are used to expected net gain on the validation set
- **Profit Aware Thresholding:**
 - Final approval threshold on expected net gain was chosen to maximize total expected net gain on the validation set
 - During decision making, probabilities are computed, re-calibrated and the chosen threshold is applied to decide whether a loan is to be given or not

Threshold Selection for Cost-Sensitive Decisions

- Suppose: Cost of false positive = C_{FP} | Cost of false negative = C_{FN}
- **Decision Rule:** Predict class 1 when expected cost of predicting 1 is lower than expected cost of predicting 0
- **Assumptions:**
 - predicted probability is trusted
 - each decision is taken independently
 - costs are per-sample and fixed
 - correct decisions have zero cost
- **Threshold Formula for Binary Classification:**

$$P(y=1|x) = \hat{p} \quad \text{Expt Cost of false positive} = (1 - \hat{p}) C_{FP}$$

$$P(y=0|x) = 1 - \hat{p} \quad \text{Expt Cost of false negative} = \hat{p} C_{FN}$$

$$\text{Decision: } (1 - \hat{p}) C_{FP} < \hat{p} C_{FN}$$

$$\hat{p}_{TH} > \frac{C_{FP}}{C_{FP} + C_{FN}}$$

Threshold Selection Example

- Consider a model built to detect potential frauds
- Suppose:
 - Fraud probability = $\hat{p}$
 - Loss if fraud is missed = L
 - Cost of investigation = C
- **Decision Rule:** Investigate only if the cost of investigation justified the loss if fraud is missed

$$\hat{p} \geq \frac{C}{L+C}$$

Decision Thresholds: Theory to Practice

- Theoretical threshold formulas are derived under simplified assumptions.
- In practice, businesses often choose the threshold by maximizing **total expected gain / profit / revenue / utility** over the validation set
- Then the best practical threshold may differ because of:
 - imperfect calibration
 - model ranking errors
 - class imbalance
 - non-uniform gains and losses across samples
 - Business trade-offs
- ✓ **Key Takeaway:** In real applications, threshold selection is often a validation-based optimization problem, not just a formula substitution

Summary

- ✓ A classifier may rank samples well, yet its scores may not be reliable probabilities.
- ✓ Model is calibrated when predicted probabilities should match observed frequencies across score ranges.
- ✓ Miscalibration can arise due to various reasons such as objective mismatch, regularization, model family, class imbalance, etc.
- ✓ Calibration can be assessed using: Brier score for overall probability accuracy and Calibration plot / reliability diagram for visual diagnosis
- ✓ Re-calibration methods: Platt scaling and Isotonic regression improve probability estimates
- ✓ Even after calibration, decision thresholding is needed because the default threshold of 0.5 is often not optimal
- ✓ Thresholds should reflect costs, losses, resource limits, or business value, not only classification error
- ✓ Good ML decisions require both well-calibrated probabilities and application-aware threshold selection

```
operation == "MIRROR_X":  
    mirror_mod.use_x = True  
    mirror_mod.use_y = False  
    mirror_mod.use_z = False  
operation == "MIRROR_Y":  
    mirror_mod.use_x = False  
    mirror_mod.use_y = True  
    mirror_mod.use_z = False  
operation == "MIRROR_Z":  
    mirror_mod.use_x = False  
    mirror_mod.use_y = False  
    mirror_mod.use_z = True
```

```
#selection at the end -add  
mirror_ob.select= 1  
modifier_ob.select=1  
context.scene.objects.active  
= ("Selected" + str(modifier_ob.name))  
mirror_ob.select = 0  
= bpy.context.selected_objects  
data.objects[one.name].select  
print("please select exactly one mirror")
```

WILLIAM C. LEE

```
def mirror(modifier):  
    #add mirror to the selected  
    #object -mirror_x  
    mirror_x
```

THANK YOU